Question 1: What is key debouncing? Explain ways to avoid it with appropriate illustration of the mechanism.

Key debouncing is the process of removing unwanted multiple signals caused by a mechanical switch. When a key is pressed or released, its contacts bounce for a few milliseconds, creating rapid changes between logic 0 and 1 instead of a single clean signal. This may cause a microcontroller to detect one press as multiple presses.

To avoid this, two main methods are used:

Software Debouncing: The system detects a key press, waits for a short delay (about 10–20 ms), and then checks the input again. If the signal is still stable, it is accepted as a valid press.

Hardware Debouncing: Electronic circuits such as an S-R latch are used to stabilize the signal. The latch holds a steady output even if the input fluctuates due to bouncing, producing a clean single transition.

Question 2: Write down the mechanism of key identification of a Hex Keyboard.

A hex keyboard (4×4 matrix keypad) uses a scanning process to detect which key is pressed. This process is performed in three main steps: column identification, row identification, and key identification.

1. Column Identification

At the beginning, all row lines are set to LOW (0), and all column lines are kept HIGH (1). In this condition, if no key is pressed, the column input reads 1111 (0FH). When a key is pressed, it connects a column line to a LOW row line, causing one column bit to become 0. For example, if the column pattern becomes

1101, it indicates that the key belongs to Column 2. The microprocessor continuously checks the column lines. If the reading is not 1111, it detects that a key has been pressed. A short delay is usually added here to handle key debouncing.

2. Row Identification

After detecting the active column, the system identifies the row by scanning each row one at a time. In this step, one row is set to LOW (0) while the others are kept HIGH (1). For example, the system may first apply 1110 to activate Row 0. Then it reads the column lines again. If the column pattern changes (not 1111), the pressed key is in that row. If the column pattern remains 1111, the system moves to the next row and repeats the process. This continues until the correct row is found.

3. Key Identification

Once both the row and column are identified, the exact key is determined. The system uses a predefined lookup table that stores the key values for each row. For example, a row may contain keys like 8, 9, A, B. The system then checks which column bit is 0 (using shift or masking operations) to find the exact position. By combining the row and column information, the microprocessor identifies the pressed key and stores its value.

Thus, by scanning columns first, then rows, and finally mapping their intersection, the hex keyboard accurately detects which key has been pressed.

Question 3: Imagine you are trying to display your student ID using 7 segment displays. Illustrate the configuration of the 7 segment

displays and explain the working mechanism using BCD to seven segment display configuration.

All segment pins (a-g) of all displays are connected in parallel to one data port of the microcontroller. Each digit has a separate enable pin connected through a transistor to another control port. Thus, one port sends segment data, and another selects the active digit.

The microcontroller sends 4-bit BCD data to a BCD-to-7-segment decoder, which generates the correct segment signals for each digit. For example,
2 → 0010, 0 → 0000, 1 → 0001, 7 → 0111, 8 → 1000
The microcontroller sends the BCD of each digit and activates one display at a time in sequence: 2 → 2 → 2 → 0 → 1 → 7 → 8 → 2
This scanning happens very fast, so all digits appear ON at the same time.

Question 4: Describe how you can reduce the pin required to connect a LED Matrix display.

An 8×8 LED matrix display normally needs 16 pins, because it has 8 row lines and 8 column lines. However, this pin requirement can be reduced using Johnson Counter method for row selection. In this approach, the 8 column pins are directly connected to an 8-bit microcontroller port and are used to send the LED pattern for each row. For the rows, instead of using 8 separate pins, a Johnson Counter is used, which typically requires only 2 control signals (clock and enable). The counter drives 8 transistor switches, where each output activates one row at a time.

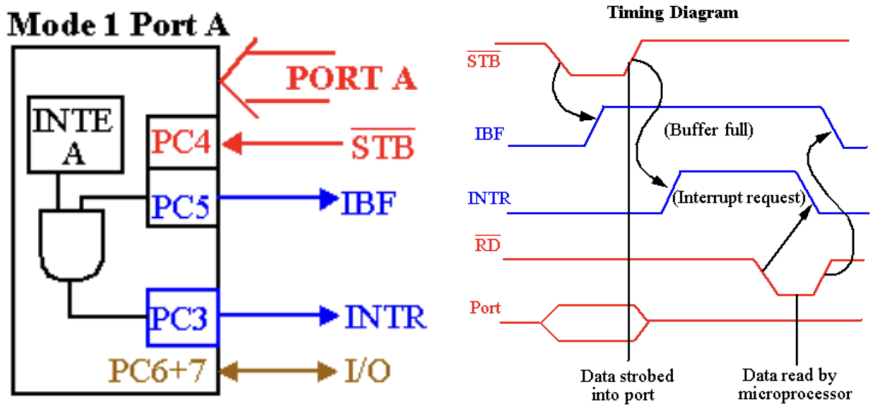
The system works by scanning rows one by one. The counter activates a row, the column data is sent for that row, and then it quickly moves to the next row. This process repeats continuously. Because the scanning is very fast, all rows appear to be ON at the same time, producing a stable image while reducing the number of required pins.

Question 5: An 82C55 IC's Port A is connected to a pendrive and port B is connected to a display device.

a. Write down the control bits for the given connection of 82C55. Assume that pendrive will send data to the CPU.

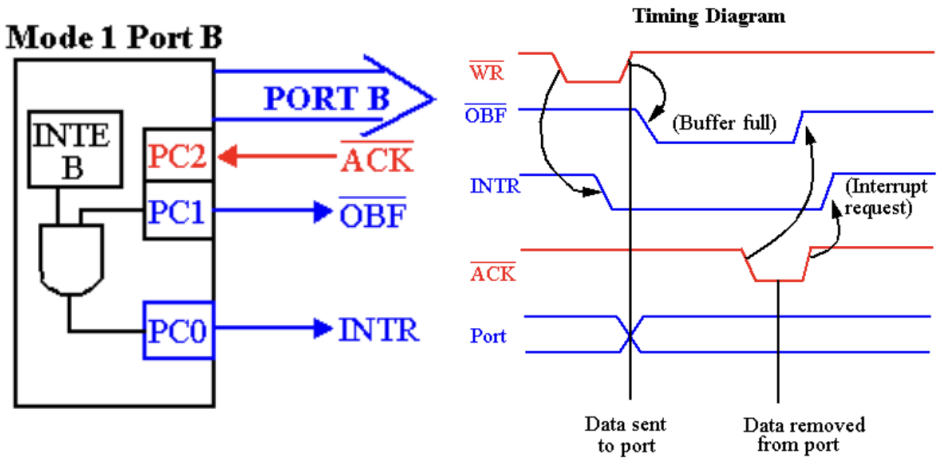
Control bits 10111100

b. Draw the handshaking diagram of port A only for the given scenario with appropriate pin numbers and signal labeling and explain the handshaking process.



The pendrive places data on Port A and sends STB (low) to indicate valid data. The 82C55 latches the data, sets IBF high, and generates INTR to notify the CPU. The CPU reads the data using RD, after which IBF and INTR are reset.

c. Draw the handshaking diagram of port B only for the given scenario with appropriate pin numbers and signal labeling and explain the handshaking process.



The CPU writes data to Port B, and the 82C55 sets OBF (low) to indicate valid data. The display reads the data and sends ACK (low). Then the 82C55 resets OBF and generates an INTR to request the next data.

d. Is there any pin left to connect a low level input device to the 82C55?

PC6, PC7 remain unused and they can be used to connect a low-level input device.

e. Describe the state of the 82C55 if the control bit is "10111010"

D7 = 1 → I/O mode

D6–D5 = 01 → Port A, Mode 1

D4 = 1 → Port A, input

D3 = 1 → Port C upper, input

D2 = 1 → Port B, Mode 1

D1 = 0 → Port B, output

D0 = 0 → Port C lower, output

So, both port A and port B are in Mode 1. Port C is splitted between input (upper) and output (lower)